

A Comparative and Visual Analysis of Exact and Heuristic Algorithms for the Knapsack Problem

APPENDIX

Name: Aarav Bhatnagar - Centre Number: 39253 - Candidate Number: 8015

Algorithms.py

```
#Here are the imported libraries and functions that I will be using
import time
import random
import heapq
import math
from matplotlib.figure import Figure
import numpy as np
from matplotlib.lines import Line2D
from collections import defaultdict

class Particle:
    def __init__(self, n):
        self.position = [random.randint(0, 1) for x in range(n)] #represents a
possible solution
        self.velocity = [random.random() for x in range(n)] #the velocity of a
particle which is initially randomised
        self.pbest = self.position[:] #the personal best solution seen
        self.pbest_fitness = 0 #the value of the personal best solution
        #Used for the graph/visualisation
        self.pbest_list = []
        self.velocity_list = []

class BinaryKP:
    def __init__(self, weights, values, capacity, algorithm_data):
        #Constant terms between each algorithm
        self.weights = weights
        self.values = values
        self.capacity = capacity
        self.algorithm_data = algorithm_data #This achieves 5.2 Algorithm Parameter
objective
        #algorithm_data is the dictionary from the input form and index.py
        #Used for easy access in the recursive algorithm
        self.recursion_counter = 0
        self.hits = 0

    def solve_01_knapsack_BF_iterative(self):
        #graph max_length
        max_length = 10000
        #Loop through all subsets/combinations of items and return one with the
highest total value that doesn't exceed self.capacity
        #A perhaps interesting way of doing this is counting upwards in binary
        #In order to count up efficiently, we must use gray codes so that between
each iteration we only flip one bit, thereby the O(n) evaluate function is not called
n times.
```

```

n = len(self.weights)
max_value = 0
best_mask = 0 # to store the Gray code subset

total_weight = 0
total_value = 0

best_values = []
current_values = []
iterations_list = []

iterations = min((2**n), self.algorithm_data['max_iterations_iterative'] + 1)
# Calculate step size to reduce list to <= max_length
step = max(1, (iterations - 1) // max_length)

for mask in range(1, iterations):#max loops from 1 to (2**n) - 1
    gray = mask ^ (mask >> 1) # compute Gray code i.e. 1110

    # Previous Gray code
    prev_gray = (mask - 1) ^ ((mask - 1) >> 1) #sub (mask - 1) into formula
i.e. 1010
    # Bit that changed
    diff = gray ^ prev_gray #XOR because 1110 1010 gives 0100
    i = n - ((diff).bit_length()) # index of changed bit (bit length returns
the number of bits to store number there 00010 returns 2)

    if gray & diff != 0: # item added prev_gray = 001, gray = 011 therefore
diff = 010: gray & diff = 011 & 010 = 010 which is not zero => item is added
        total_weight += self.weights[i]
        total_value += self.values[i]
    else: # item removed
        total_weight -= self.weights[i]
        total_value -= self.values[i]

    if total_weight <= self.capacity and total_value > max_value:
        max_value = total_value
        best_mask = gray # store best subset mask

    #Due to the significant amount of iterations, I decided to only record
data for the graph after a fixed number of steps
    if mask % step == 0:
        best_values.append(max_value)
        current_values.append(total_value if total_weight <= self.capacity
else 0)

        iterations_list.append(mask)

```

```

end_time = time.perf_counter()
fig = Figure(dpi=250)
axis = fig.add_subplot(1, 1, 1)

axis.plot(iterations_list, best_values, marker='o', label='Best Value')
axis.plot(iterations_list, current_values, label='Current Value', alpha=0.5,
color='grey')

axis.set_title('Brute Force (iterative) - 0/1 Knapsack Problem')
axis.set_xlabel('Iterations')
axis.set_ylabel('Value')
axis.legend()

# Format mask as binary string (n bits, left to right = item 0..n-1)
return max_value, bin(best_mask)[2:].zfill(n), fig, end_time

#Recursively called plotting methods
def plot_node(self, axis, x, y, lastnode=False):
    node_size = 10
    axis.scatter(x, y, s=node_size, color='black', label='node', zorder=2)

    if lastnode:
        #If it is a last node, the branch will end and display a diamond rather
than a circle
        axis.scatter(x, y-0.25, s=2, color='black', marker='d', alpha=0.5,
label='hidden nodes')

def solve_01_knapsack_BF_recursive(self):
    fig = Figure(dpi=250)
    axis = fig.add_subplot(1, 1, 1)
    axis.axis('off')
    axis.set_title('Brute Force (recursive) - 0/1 Knapsack Problem')

    #Number of layers shown on the decision tree
    max_tree_layer = 7

    items = len(self.weights)

    #used for the decision tree when selecting the corret object to be
highlighted
    path_edges = defaultdict(dict)
    #inner recursive function
    def solve(capacity=0, n=0, best_combination='', plot_pos_x=0, plot_pos_y=0,
dx=20):
        if n == 0:
            capacity = self.capacity

```

```

        self.recursion_counter += 1
        if n == max_tree_layer and items > max_tree_layer and plot_pos_y != 0:
self.plot_node(axis, plot_pos_x, plot_pos_y, lastnode=True)
        elif n <= max_tree_layer:
            #Call the plot node function which adds to the figure the marked plot
at a given x, y coord
            self.plot_node(axis, plot_pos_x, plot_pos_y)

        if self.recursion_counter >=
self.algorithm_data['max_iterations_recursive']:
            return (0, '')

        if n == len(self.weights) or capacity == 0:
            #Adds '0' to the end of the string in order to ensure it is the
specified length
            return (0, best_combination.ljust(len(self.weights), '0'))
        else:
            including_item = 0
            if self.weights[n] <= capacity:
                #including item
                xi, yi = plot_pos_x - dx, plot_pos_y - 1
                if n < max_tree_layer:
                    #Plots a line between previous node and new node position
                    line = axis.plot([plot_pos_x, xi], [plot_pos_y, yi],
color="green", label="included", zorder=1)
                    path_edges[n][best_combination + '1'] = line
                    including_item, best_combination1 = solve(capacity-
self.weights[n], n+1, best_combination + '1', xi, yi, dx / 2)
                    including_item += self.values[n]

                #excluding item
                xe, ye = plot_pos_x + dx, plot_pos_y - 1
                if n < max_tree_layer:
                    #Plots a line between previous node and new node position
                    line = axis.plot([plot_pos_x, xe], [plot_pos_y, ye], color="red",
label="excluded", zorder=1)
                    path_edges[n][best_combination + '0'] = line
                    excluding_item, best_combination2 = solve(capacity, n+1,
best_combination + '0', xe, ye, dx/2)

                if including_item > excluding_item:
                    return (including_item, best_combination1)
                else:
                    return (excluding_item, best_combination2)

```

```

value, solution = solve()

end_time = time.perf_counter()

#highlight selected path
for depth in range(min(max_tree_layer, items)):
    prefix = solution[:depth + 1]
    line = path_edges[depth].get(prefix)
    if line:
        #Coloured blue and labelled select
        line[0].set_color('blue')
        line[0].set_label('selected')

#In the table I will show the number of recursive calls
display_text = "Number of recursive calls: " + str(self.recursion_counter) +
(" (limit exceeded)" if self.recursion_counter >=
self.algorithm_data['max_iterations_recursive'] else '')
axis.text(
    0.5, 0, display_text,
    transform=axis.transAxes,
    fontsize=12,
    fontweight="bold",
    va="center",
    ha="center",
)

#The decision tree will be incomplete if the number of recursive calls
exceeds the recursion limit
#This is because not all branches will be explored

lines, labels = axis.get_legend_handles_labels()
# Merge them, removing duplicates by label
by_label = dict(zip(labels, lines))

axis.legend(by_label.values(), by_label.keys(), loc='upper right')

return value, solution, fig, end_time

def solve_01_knapsack_GApprox(self, weights=[], values=[], capacity=[],
get_total_weight=False, generate_graph=False):
    #To get a general approximation (this would be exact for the fractional
    knapsack problem), we can generate a list of value / weight
    if len(weights) == 0:
        weights = self.weights
        values = self.values
        capacity = self.capacity

```

```

n = len(weights)
items = [None] * n
for x in range(n):
    weight = weights[x]
    value = values[x]
    items[x] = ((value/weight, weight, value, x))
items = sorted(items, reverse=True)
#Sorts in descending order
total_weight = 0
max_value = 0
best_combination = ['0'] * n
final_item_index = 0
#includes item until the current item weight + total weight > capacity
for x in range(n):
    if items[x][1] + total_weight > capacity:
        final_item_index = x
        break
    total_weight += items[x][1]
    max_value += items[x][2]
    best_combination[items[x][3]] = '1'

if generate_graph:
    #Only called when not used as bound or starting solution
    #Graph generated is ordered by the items selected not the actual item
    index

    end_time = time.perf_counter()
    fig = Figure(dpi=250)
    axis = fig.add_subplot(1, 1, 1)

    item_list = list(range(n))
    item_list_1 = item_list[:final_item_index]
    item_list_2 = item_list[final_item_index:]

    axis.scatter(item_list_1, [items[x][0] for x in range(final_item_index)],
color='green', label='included')
    axis.scatter(item_list_2, [items[x][0] for x in range(final_item_index,
n)], color='red', label='excluded')

    axis.set_title('Greedy Approximation - 0/1 Knapsack Problem')
    axis.set_xlabel('Items Sorted')
    axis.set_ylabel('Value/Weight Ratio')
    axis.legend()
    #greedy graph will have
    return max_value, "".join(best_combination), fig, end_time
else:

```

```

        if get_total_weight:
            return max_value, "".join(best_combination), total_weight
        else:
            return max_value, "".join(best_combination)

def solve_fractional_knapsack_G(self, weights, values, capacity):
    n = len(weights)
    items = []
    #Same as in solve_01_knapsack_GApprox()
    for x in range(n):
        weight = weights[x]
        value = values[x]
        items.append((value/weight, weight, value))
    items = sorted(items, reverse=True)
    total_weight = 0
    max_value = 0
    for x in range(n):
        if items[x][1] + total_weight > capacity:
            max_value += (items[x][2] * ((capacity - total_weight)/items[x][1]))
            #Includes the maximum fractional amount of this last item
            break
        total_weight += items[x][1]
        max_value += items[x][2]

    return max_value

def solve_01_knapsack_DP_Top_Down(self):
    memo = {}
    memo_size = []
    hits_list = []
    #follows the same recursive idea as in solve_01_knapsack_BF_recursive()
    def solve(capacity=0, n=0, best_combination=''):
        self.recursion_counter += 1
        memo_size.append(len(memo))
        hits_list.append(self.hits)
        if n == 0:
            capacity = self.capacity

        if n == len(self.weights):
            return (0, '')
        elif capacity == 0:
            return (0, '0' * (len(self.weights) - n - 1))
        elif (capacity, n) in memo:
            self.hits += 1
            return memo[(capacity, n)]
        else:

```

```

        including_item = 0
        if self.weights[n] <= capacity:
            including_item, best_combination1 = solve(capacity-
self.weights[n], n+1, best_combination)
            including_item += self.values[n]

        excluding_item, best_combination2 = solve(capacity, n+1,
best_combination)

        if including_item > excluding_item:
            result = (including_item, '1' + best_combination1)
        else:
            result = (excluding_item, '0' + best_combination2)

        memo[(capacity, n)] = result
        return result

value, solution = solve()

end_time = time.perf_counter()
fig = Figure(dpi=250)
axis = fig.add_subplot(1, 1, 1)

recursions_list = list(range(self.recursion_counter))
axis.plot(recursions_list, memo_size, label='Memo Size')
axis.set_title('DP (Memoization) - 0/1 Knapsack Problem')
axis.set_xlabel('Recursive calls')
axis.set_ylabel('Count')
axis.plot(recursions_list, hits_list, label='hits', color="green")

axis.legend(loc='upper left')

#Displays the number of total hits achieved
axis.text(
    0.75, 0.10, 'Total hits: ' + str(self.hits),
    transform=axis.transAxes,
    fontsize=12,
    fontweight="bold",
    va="center",
    ha="center",
)

return value, solution, fig, end_time

def solve_01_knapsack_DP_Bottom_Up(self):
    #Bottom up (Tabulation)

```

```

n = len(self.weights)
#Rows are items and columns are self.capacity
#dp[i][c] = best value using items 0...row-1 with self.capacity x
#therefore columns is capacity and rows is number of items
dp = [[0] * (self.capacity + 1) for _ in range(n+1)]
# Fill table bottom-up
for i in range(1, len(dp)):#i-items
    for c in range(self.capacity + 1):#c-self.capacity
        if self.weights[i-1] <= c:
            include_item = dp[i-1][c-self.weights[i-1]] + self.values[i-1]#row above shifted along by the weight of the new item
            exclude_item = dp[i-1][c] #row above
            dp[i][c] = max(include_item, exclude_item)
        else:
            dp[i][c] = dp[i-1][c] #Carry over from row above
max_value = dp[n][self.capacity]

#Reconstruct best_combination
i = n
c = self.capacity
best_combination = ['0'] * n
while i != 0 and c != 0:
    if dp[i][c] != dp[i-1][c]:#Different value, therefore item was picked
        best_combination[i-1] = '1'
        c -= self.weights[i-1]
        i -= 1
    else:
        i -= 1

end_time = time.perf_counter()

#Crops data into a labelled table to be displayed on output
m_data = np.array(dp)
items_list = np.array(list(range(n + 1)))
m_data = np.column_stack((items_list, m_data))

capacity_list = np.array([0] + list(range(self.capacity + 1)))
m_data = np.vstack((capacity_list, m_data))

data = m_data[:,250, :250]

return max_value, ''.join(best_combination), data, end_time

def solve_01_knapsack_BnB(self):
    n = len(self.weights)
    max_value = 0

```

```

best_solution = '0' * n
priority_queue = []
priority_queue_length_list = []
branches_pruned = 0
branches_pruned_list = []
# Compute initial bounds
ub_root = math.floor(self.solve_fractional_knapsack_G(self.weights,
self.values, self.capacity))
lb_root, lb_root_solution = self.solve_01_knapsack_GApprox()

# Priority queue stores (-ub, lb, weight, value, solution, index,
lb_solution)
# Negate ub to simulate max-heap (heapq defaults to minheap). Therefore those
with a maximum ub are explored first.
# Algorithm should trap ub and lb in order to find optimal solution
heapq.heappush(priority_queue, (-ub_root, lb_root, 0, 0, '', 0,
lb_root_solution))
iterations = 0
while priority_queue:
    iterations += 1
    priority_queue_length_list.append(len(priority_queue))
    branches_pruned_list.append(branches_pruned)
    #pop the next node from the queue
    neg_ub, lb, current_weight, current_value, current_solution, i,
lb_solution = heapq.heappop(priority_queue)
    ub = -neg_ub

    #End of branch
    if i >= n:
        continue

    #Prune branches
    if current_weight > self.capacity or ub <= max_value:
        branches_pruned += 1
        continue

    #set max_value
    if lb > max_value:
        max_value = lb
        best_solution = current_solution + lb_solution

    #including item
    new_weight = current_weight + self.weights[i]
    if new_weight <= self.capacity:
        new_value = current_value + self.values[i]
        remaining_capacity = self.capacity - new_weight

```

```

        #only lb is an actual solution
        ub_incl = new_value +
math.floor(self.solve_fractional_knapsack_G(self.weights[i+1:], self.values[i+1:],
remaining_capacity))
        lb_incl, lb_incl_solution =
self.solve_01_knapsack_GApprox(self.weights[i+1:], self.values[i+1:],
remaining_capacity)
        lb_incl += new_value
        #only add to the queue to be explored if the ub is greater than the
max value
        if ub_incl > max_value:
            heapq.heappush(priority_queue, (-ub_incl, lb_incl, new_weight,
new_value, current_solution + '1', i + 1, lb_incl_solution))

        #excluding item
        ub_excl = current_value +
math.floor(self.solve_fractional_knapsack_G(self.weights[i+1:], self.values[i+1:],
self.capacity - current_weight))
        lb_excl, lb_excl_solution =
self.solve_01_knapsack_GApprox(self.weights[i+1:], self.values[i+1:], self.capacity -
current_weight)
        lb_excl += current_value
        if ub_excl > max_value:
            heapq.heappush(priority_queue, (-ub_excl, lb_excl, current_weight,
current_value, current_solution + '0', i + 1, lb_excl_solution))

end_time = time.perf_counter()
fig = Figure(dpi=250)
axis = fig.add_subplot(1, 1, 1)

iterations_list = list(range(iterations))
axis.plot(iterations_list, priority_queue_length_list, label='Priority Queue
Length')
axis.set_title('Branch and Bound - 0/1 Knapsack Problem')
axis.set_xlabel('Iterations')
axis.set_ylabel('Length')

axis2 = axis.twinx()
axis2.set_ylabel('Branches Pruned')
axis2.plot(iterations_list, branches_pruned_list, label='Branches Pruned',
color="green")

# Optional: Add legends for both axes
lines1, labels1 = axis.get_legend_handles_labels()
lines2, labels2 = axis2.get_legend_handles_labels()
axis.legend(lines1 + lines2, labels1 + labels2, loc='upper right')

```

```

        return max_value, best_solution, fig, end_time

def solve_01_knapsack_GeneticAlg(self):
    #variables
    n = len(self.weights)

    isTournamentSelection = True if self.algorithm_data['selection_type_GA'] ==
    "Tournament selection" else False
    tournament_size = self.algorithm_data['tournament_size_GA']

    GENERATIONS = self.algorithm_data['max_iterations_GA']
    population_size = self.algorithm_data['population_size_GA']
    crossover_rate = self.algorithm_data['cross_over_rate']
    mutation_rate = self.algorithm_data['mutation_rate']
    elitism_rate = self.algorithm_data['elitism_rate']

    elitism_number = int(elitism_rate * population_size)
    global_best_combination = '0' * n
    global_max_value = 0

    population = [''] * population_size
    fitnesses = [0] * population_size
    #Graph lists
    average_fitnesses = [0] * GENERATIONS
    best_fitnesses = [0] * GENERATIONS
    lowest_fitnesses = [0] * GENERATIONS
    #Generate random population to begin
    for x in range(population_size):
        value, binary_string, weight =
self.generate_starting_solution(is_greedy=False)
        population[x] = binary_string
        fitnesses[x] = value

    for i in range(GENERATIONS):
        new_population = [''] * population_size
        if isTournamentSelection:
            #Tournament Selection (else roulette wheel selection)
            for x in range(population_size):
                binary_string = '0' * n
                max_value = 0
                for y in range(tournament_size):
                    index = random.randint(0, population_size - 1)
                    if fitnesses[index] > max_value:
                        binary_string = population[index]
                        max_value = fitnesses[index]

```

```

        new_population[x] = binary_string
    else:
        total_fitness = sum(fitnesses)
        for x in range(population_size):
            random_variable = random.random() * total_fitness
            cumulative_sum = 0
            solution_index = 0
            for p in range(population_size):
                cumulative_sum += fitnesses[p]
                if random_variable <= cumulative_sum:
                    solution_index = p
                    break
            new_population[x] = population[solution_index]

#Cross over certain pairs of the population
for x in range(0, population_size-1, 2):
    cross_over_random_variable = random.random()
    if cross_over_random_variable > crossover_rate: continue
    crossover_point = random.randint(0, n-1)

    new_population[x], new_population[x+1] = new_population[x]
[:crossover_point] + new_population[x+1][crossover_point:], new_population[x+1]
[:crossover_point] + new_population[x][crossover_point:]

#Mutate to introduce variance
for x in range(population_size):
    for mutation_bit in range(n):
        mutate_random_variable = random.random()
        if mutate_random_variable > mutation_rate: continue
        new_population[x] = (
            new_population[x][:mutation_bit] +
            ('1' if new_population[x][mutation_bit] == '0' else '0') +
            new_population[x][mutation_bit+1:]
        )

#Elitism
#Uses heap data structure to efficiently find the n largest item indexes
indexes = heapq.nlargest(elitism_number, range(len(fitnesses)),
key=fitnesses.__getitem__)
for x in range(elitism_number):
    swapping_index = random.randint(0, population_size-1)
    new_population[swapping_index] = population[indexes[x]]

average_fitnesses[i] = sum(fitnesses) / population_size
best_fitnesses[i] = max(fitnesses)
if best_fitnesses[i] > global_max_value:

```

```

        global_max_value = best_fitnesses[i]
        global_best_combination =
population[fitnesses.index(global_max_value)]
        lowest_fitnesses[i] = min(x for x in fitnesses if x > 0)

        population = new_population
        fitnesses = list(map(self.evaluate, population))

    end_time = time.perf_counter()
    fig = Figure(dpi=250)
    axis = fig.add_subplot(1, 1, 1)

    generations = list(range(GENERATIONS))

    #Plot average, best and minimum fitness all on same graph
    axis.plot(generations, average_fitnesses, marker='o', label='Average
Fitness')
    axis.plot(generations, best_fitnesses, marker='o', label='Highest Fitness')
    axis.plot(generations, lowest_fitnesses, marker='o', label='Minimum Fitness')

    axis.set_title('Genetic Algorithm - 0/1 Knapsack Problem')
    axis.set_xlabel('Iterations')
    axis.set_ylabel('Fitness')
    axis.legend()

    return global_max_value, global_best_combination, fig, end_time

def solve_01_knapsack_random(self):
    n = len(self.weights)
    RUNTIME = self.algorithm_data['time_limit']
    time_interval = min(0.001, RUNTIME / 100)
    start_time = last_time = time.time()
    best_combination = '0' * n
    max_value = 0

    best_values = []
    current_values = []
    times_list = []
    counter = 0
    while time.time() < start_time + RUNTIME:
        #Generates a random binary string of fixed length n
        binary_string = bin(random.randint(0, (2**n)-1))[2:].zfill(n)

        value = self.evaluate(binary_string)
        if value > max_value:
            best_combination = binary_string

```

```

        max_value = value

    if time.time() - last_time >= time_interval:
        last_time = time.time()
        best_values.append(max_value)
        current_values.append(value)
        times_list.append(last_time - start_time)

end_time = time.perf_counter()
fig = Figure(dpi=250)
axis = fig.add_subplot(1, 1, 1)

axis.plot(times_list, best_values, marker='o', label='Best Value')
axis.plot(times_list, current_values, label='Current Value', alpha=0.5,
color='red')

axis.set_title('Randomised Search - 0/1 Knapsack Problem')
axis.set_xlabel('Time')
axis.set_ylabel('Value')
axis.legend()

return max_value, best_combination, fig, end_time

def solve_01_knapsack_SA(self):
    n = len(self.weights)
    value, solution, weight =
self.generate_starting_solution(self.algorithm_data['is_greedy_SA'])

    max_value = value
    best_solution = solution

    initial_temperature = self.algorithm_data['initial_temperature']
    minimum_temperature = self.algorithm_data['minimum_temperature']
    alpha = self.algorithm_data['alpha']

    iteration = 0
    max_iterations = self.algorithm_data['max_iterations_SA']

    #worse = 0
    #accepted = 0

    range_of_data = max(self.values) - min(self.values)

    temperature = initial_temperature

    temperature_list = []

```

```

best_values = []
current_values = []

while temperature >= minimum_temperature and iteration < max_iterations:
    temperature_list.append(temperature)
    Found = False
    while Found == False:
        random_bit = random.randint(0, n-1)
        if solution[random_bit] == '1':
            new_solution = solution[:random_bit] + ('0') +
solution[random_bit + 1:]
            new_value = value - self.values[random_bit]
            new_weight = weight - self.weights[random_bit]
            Found = True
        else:
            new_solution = solution[:random_bit] + ('1') +
solution[random_bit + 1:]
            new_value = value + self.values[random_bit]
            new_weight = weight + self.weights[random_bit]

            if new_weight <= self.capacity:
                Found = True

    if new_value >= value:
        value = new_value
        weight = new_weight
        solution = new_solution

    if new_value > max_value:
        max_value = new_value
        best_solution = new_solution
    else:
        #Probability formula
        #Delta will normalise the range as input data may vary in terms of
'Range of values:' variable
        delta = (value - new_value) / range_of_data #relative positive
'worse' difference
        acceptance_probability = math.exp(-delta / temperature)
        #print("Difference: " + str(value-new_value) + "acceptance
probability: " + str(acceptance_probability))
        rand_var = random.random()

        #worse += 1
        if rand_var < acceptance_probability:
            value = new_value
            weight = new_weight

```

```

        solution = new_solution
        #accepted += 1

    best_values.append(max_value)
    current_values.append(value)

    temperature *= alpha
    iteration += 1
    #print(accepted, worse, accepted/worse, iteration, temperature, value,
max_value)

    # fig will include cooling schedule against iterations, current_solution and
best_solution, greedy if there
    end_time = time.perf_counter()
    fig = Figure(dpi=250)
    axis = fig.add_subplot(1, 1, 1)
    iterations_list = list(range(iteration))

    axis.set_title('Simulated Annealing - 0/1 Knapsack Problem')
    axis.set_xlabel('Iterations')
    axis.set_ylabel('Value')
    axis.plot(iterations_list, best_values, marker='o', label='Best Value')
    axis.plot(iterations_list, current_values, label='Current Value', alpha=0.5,
color='grey')

    axis2 = axis.twinx()
    axis2.set_ylabel('Temperature')
    axis2.plot(iterations_list, temperature_list, label='Temperature',
color="red")

    #Add legends for both axes
    lines1, labels1 = axis.get_legend_handles_labels()
    lines2, labels2 = axis2.get_legend_handles_labels()
    axis.legend(lines1 + lines2, labels1 + labels2, loc='upper right')

    return max_value, best_solution, fig, end_time

def solve_01_knapsack_Tabu(self):
    n = len(self.weights)
    value, solution, weight =
self.generate_starting_solution(self.algorithm_data['is_greedy_tabu'])

    max_value = value
    best_solution = solution

    tabu_list = []

```

```

tabu_list_maxsize = self.algorithm_data['tabu_list_size']
ITERATIONS = self.algorithm_data['max_iterations_tabu_search']

best_values = []
current_values = []
tabu_length_list = []

for y in range(ITERATIONS):
    #Efficiently finding next move/neighbour by evaluating possible new
    solutions and selecting the best non-tabu solution
    best_neighbour = ''
    best_neighbour_weight = 0
    best_neighbour_value = 0
    worse = False
    for x in range(n):
        #There are n neighbours
        if solution[x] == '0':
            #We flip to 1
            if weight + self.weights[x] <= self.capacity:
                new_neighbour_value = value + self.values[x]
                new_neighbour = solution[:x] + '1' + solution[x+1:]
                new_neighbour_weight = weight + self.weights[x]
                if new_neighbour_value > best_neighbour_value and
int(new_neighbour, 2) not in tabu_list:
                    best_neighbour_value = new_neighbour_value
                    best_neighbour = new_neighbour
                    best_neighbour_weight = new_neighbour_weight
            elif solution[x] == '1':
                #We flip to 0
                new_neighbour_value = value - self.values[x]
                new_neighbour = solution[:x] + '0' + solution[x+1:]
                new_neighbour_weight = weight - self.weights[x]
                if new_neighbour_value > best_neighbour_value and
int(new_neighbour, 2) not in tabu_list:
                    best_neighbour_value = new_neighbour_value
                    best_neighbour = new_neighbour
                    best_neighbour_weight = new_neighbour_weight
                    worse = True

    if best_neighbour_value > max_value:
        max_value = best_neighbour_value
        best_solution = best_neighbour
        if worse: print("accepted worse " + str(y))

value = best_neighbour_value
solution = best_neighbour

```

```

weight = best_neighbour_weight

tabu_list.append(int(solution, 2))

while len(tabu_list) > tabu_list_maxsize:
    tabu_list.pop(0)

best_values.append(max_value)
current_values.append(value)
tabu_length_list.append(len(tabu_list))

end_time = time.perf_counter()
#In the graph for tabu search we will include the length of the tabu list,
the max value, the current value as they are the relevant properties
fig = Figure(dpi=250)
axis = fig.add_subplot(1, 1, 1)
iterations_list = list(range(ITERATIONS))
axis.set_title('Tabu Search - 0/1 Knapsack Problem')
axis.set_xlabel('Iterations')
axis.set_ylabel('Value')
axis.plot(iterations_list, best_values, marker='o', label='Best Value')
axis.plot(iterations_list, current_values, label='Current Value', alpha=0.5,
color='grey')

axis2 = axis.twinx()
axis2.set_ylabel('Length')
axis2.plot(iterations_list, tabu_length_list, label='Length of Tabu List',
color="green")

# Optional: Add legends for both axes
lines1, labels1 = axis.get_legend_handles_labels()
lines2, labels2 = axis2.get_legend_handles_labels()
axis.legend(lines1 + lines2, labels1 + labels2, loc='lower right')

return max_value, best_solution, fig, end_time

def solve_01_knapsack_ACO(self):
    n = len(self.weights)
    ITERATIONS = self.algorithm_data['max_iterations_ACO']
    ALPHA = self.algorithm_data['alpha_ACO']
    BETA = self.algorithm_data['beta_ACO']
    number_of_ants = self.algorithm_data['ants']
    evaporation_rate = self.algorithm_data['evaporation_rate']

    global_best_combination = ['0'] * n
    max_value = 0

```

```

attractiveness = [self.values[i] / (self.weights[i]) for i in range(n)]

pheremones = [0.01] * n

best_value_list = []
pheremones_storage_list = []

for x in range(ITERATIONS):
    #Generate probabilities for each iteration
    probabilities = [0] * n
    for i in range(n):
        probabilities[i] = (pheremones[i]**ALPHA) * (attractiveness[i]**BETA)

    #Generate solution for each ant, and update pheremones for next iteration
    for y in range(number_of_ants):
        binary_string = ['0'] * n
        current_weight = 0
        current_value = 0

        feasible_set = list(range(n))

        while feasible_set:
            feasible_set = [i for i in feasible_set if current_weight +
self.weights[i] <= self.capacity]
            if not feasible_set: break

            feasible_probabilities = [probabilities[i] for i in feasible_set]
            total = sum(feasible_probabilities)

            if total == 0: break

            Random_Variable = random.random() * total
            cumulative_sum = 0
            item_index = 0

            for index, p in zip(feasible_set, feasible_probabilities):
                cumulative_sum += p
                if Random_Variable <= cumulative_sum:
                    item_index = index
                    break

            binary_string[item_index] = '1'
            current_weight += self.weights[item_index]
            current_value += self.values[item_index]
            feasible_set.remove(item_index)

```

```

        if current_value > max_value:
            global_best_combination = binary_string[:]
            max_value = current_value

    try:
        #Update pheromones
        pheromone_strength = 1 / (1 + ((max_value - current_value) /
max_value))

    except:
        continue

    for w in range(n):
        if binary_string[w] == '1':
            pheromones[w] += pheromone_strength

    best_value_list.append(max_value)
    pheromones_storage_list.append(pheromones)

    #Evapouration
    pheromones = [evapouration_rate * pheromone for pheromone in pheromones]

end_time = time.perf_counter()
#Graph for ACO will have the following components:
#Pheromone Strength, Attractiveness, probabilities, best value
fig = Figure(dpi=250)
axis = fig.add_subplot(1, 1, 1)

iterations_list = list(range(ITERATIONS))

axis.plot(iterations_list, best_value_list, marker='o', label='Best Value')

axis.set_title('Ant Colony Optimisation - 0/1 Knapsack Problem')
axis.set_xlabel('Iterations')
axis.set_ylabel('Value')

axis2 = axis.twinx()
axis2.set_ylabel('Pheromone Strength')
for x in range(n):
    axis2.plot(iterations_list, [pheromones_storage_list[y][x] for y in
range(ITERATIONS)], alpha=0.7)

# Create a proxy artist for the legend
multi_coloured_proxy = Line2D([0], [0], color='black', linewidth=2)
# Add legend manually
lines1, labels1 = axis.get_legend_handles_labels()

```

```

axis.legend(lines1 + [multi_coloured_proxy], labels1 + ["(Multicoloured)
Pheremones"], loc="upper left")

return max_value, "".join(global_best_combination), fig, end_time

def solve_01_knapsack_PS0(self):
    def repair(solution, greedy_ordering):
        #ADD phase
        total_weight = sum([self.weights[x] for x in range(len(self.weights)) if
solution[x] == 1])
        gap = self.capacity - total_weight
        i = 0
        new_solution = list(solution).copy()

        while gap > 0 and i < n:
            index = greedy_ordering[i]
            if solution[index] == 0 and gap >= self.weights[index]:
                new_solution[index] = 1
                gap -= self.weights[index]
                total_weight += self.weights[index]
            i += 1

        #DROP phase
        over = total_weight - self.capacity
        i = 0
        while over > 0 and i < n:
            index = greedy_ordering[n-1-i]
            if solution[index] == 1:
                new_solution[index] = 0
                over -= self.weights[index]
            i += 1

        return new_solution

    def evaluate_PS0(integer_list):
        #uses integer lists
        total_weight = sum([self.weights[x] for x in range(len(self.weights)) if
integer_list[x] == 1])
        total_value = sum([self.values[x] for x in range(len(self.values)) if
integer_list[x] == 1])
        return total_value if total_weight <= self.capacity else 0

    #Uses list of integers until very end
    n = len(self.weights)
    max_iterations = self.algorithm_data['max_iterations_PS0']
    population = self.algorithm_data['population_size_PS0']

```

```

inertia = self.algorithm_data['inertia']
cognitive_factor = self.algorithm_data['cognitive_factor']
social_factor = self.algorithm_data['social_factor']
steepness = self.algorithm_data['steepness']
max_velocity = self.algorithm_data['max_velocity']
best_value_list = []

swarm = [Particle(n) for _ in range(population)]
iteration = 0
gbest = []
gbest_fitness = 0

#Compute greedy ordering
items = [None] * n
for x in range(n):
    items[x] = (self.values[x]/self.weights[x], x)
items = sorted(items, reverse=True)
greedy_ordering = [b for (a, b) in items]

while iteration < max_iterations:
    for particle in swarm:
        particle.position = repair(particle.position, greedy_ordering).copy()
        new_fitness = evaluate_PSO(particle.position)

        if new_fitness > particle.pbest_fitness:
            particle.pbest = particle.position[:]
            particle.pbest_fitness = new_fitness

        if particle.pbest_fitness > gbest_fitness:
            gbest = particle.pbest[:]
            gbest_fitness = particle.pbest_fitness

        particle.pbest_list.append(particle.pbest_fitness)

    for particle in swarm:
        for x in range(n):
            #UPDATE VELOCITY
            particle.velocity[x] = (inertia * particle.velocity[x]) +
(cognitive_factor * random.random() * ((particle.pbest[x]) - (particle.position[x])))
+ (social_factor * random.random() * ((gbest[x]) - (particle.position[x])))
            particle.velocity[x] = max(min(particle.velocity[x],
max_velocity), -max_velocity)
            #print(particle.velocity[x])
            #UPDATE POSITION
            if random.random() < (1 / (1 + math.exp(-steepness *
particle.velocity[x]))):

```

```

        particle.position[x] = 1
    else:
        particle.position[x] = 0

    particle.velocity_list.append(sum(particle.velocity) /
len(particle.velocity))

    best_value_list.append(gbest_fitness)
    iteration += 1

end_time = time.perf_counter()
fig = Figure(dpi=250)
axis = fig.add_subplot(1, 1, 1)

iterations_list = list(range(max_iterations))

axis.plot(iterations_list, best_value_list, marker='o', label='Best Value')

for particle in swarm:
    axis.plot(iterations_list, particle.pbest_list, color='red', alpha=0.5,
marker='s', markersize=1, label='Personal Best Value')

axis.set_title('Particle Swarm Optimisation - 0/1 Knapsack Problem')
axis.set_xlabel('Iterations')
axis.set_ylabel('Value')

axis2 = axis.twinx()
axis2.set_ylabel('Velocity')
for particle in swarm:
    axis2.plot(iterations_list, particle.velocity_list, color='green',
label='Average Velocity')

# Get handles and labels from both axes
lines1, labels1 = axis.get_legend_handles_labels()
lines2, labels2 = axis2.get_legend_handles_labels()

# Merge them, removing duplicates by label
by_label = dict(zip(labels1 + labels2, lines1 + lines2))

axis.legend(by_label.values(), by_label.keys(), loc='center right')

return gbest_fitness, "".join(map(str, gbest)), fig, end_time

def evaluate(self, binary_string, get_total_weight=False):
    total_value = sum([self.values[x] for x in range(len(self.values)) if
binary_string[x] == "1"])

```

```

        total_weight = sum([self.weights[x] for x in range(len(self.weights)) if
binary_string[x] == "1"])

    if total_weight > self.capacity:
        total_value = 0

    if get_total_weight:
        return total_value, total_weight
    else:
        return total_value

def generate_starting_solution(self, is_greedy):
    if is_greedy:
        value, solution, weight =
self.solve_01_knapsack_GApprox(get_total_weight=True)
    else:
        n = len(self.weights)
        solution = bin(random.randint(0, (2**n)-1))[2:].zfill(n)
        value, weight = self.evaluate(solution, get_total_weight=True)
        while value == 0:
            solution = bin(random.randint(0, (2**n)-1))[2:].zfill(n)
            value, weight = self.evaluate(solution, get_total_weight=True)

    return value, solution, weight

#Function called by index.py
def time_and_run_binaryKP(function_index, weights, values, capacity, algorithm_data):
    problem = BinaryKP(weights, values, capacity, algorithm_data)
    functions = [problem.solve_01_knapsack_random,
problem.solve_01_knapsack_BF_iterative, problem.solve_01_knapsack_BF_recursive,
problem.solve_01_knapsack_GApprox, problem.solve_01_knapsack_DP_Top_Down,
problem.solve_01_knapsack_DP_Bottom_Up, problem.solve_01_knapsack_BnB,
problem.solve_01_knapsack_GeneticAlg, problem.solve_01_knapsack_ACO,
problem.solve_01_knapsack_SA, problem.solve_01_knapsack_Tabu,
problem.solve_01_knapsack_PSO]
    start_time = time.perf_counter()
    function = functions[function_index]
    if function == problem.solve_01_knapsack_GApprox:
        Answer = problem.solve_01_knapsack_GApprox(generate_graph=True)
    else:
        Answer = function()
    end_time = Answer[-1]
    elapsed_time = end_time - start_time

    fig = None
    if len(Answer) > 3:

```

```

#There is a figure
fig = Answer[2]

solution_value, solution_weight = problem.evaluate(Answer[1],
get_total_weight=True)
return Answer[0], solution_weight, Answer[1], elapsed_time, Answer[0] ==
solution_value, fig

#Function called by index.py
def generate_input_data_binaryKP(n, range_of_values, correlation_strength,
capacity_load):
    #request.form['number_of_items'], request.form['range_of_values'],
request.form['correlation_strength'], request.form['capacity_load']
    #Input data parameters
    weights = [0] * n
    values = [0] * n
    capacity = 0

    index = 0

    for index in range(n):
        weight = random.randint(1, range_of_values)
        weights[index] = weight

        if correlation_strength == 0:
            value = random.randint(1, range_of_values)
        else:
            value = max(1, min(range_of_values,
random.randint(int(correlation_strength * weight), int((2 - correlation_strength) *
weight))))

        #When correlation strength is 0 between 1 and 2W (still weakly
constrained)
        values[index] = value

    capacity = max(1, min(sum(weights), int(capacity_load * sum(weights))))

    return weights, values, capacity

```

index.py

```
#Imported libraries
from flask import *
from matplotlib.backends.backend_agg import FigureCanvasAgg as FigureCanvas
import pandas as pd
import math
import ast
import io
import base64
import numpy
import json

app = Flask(__name__)

app.secret_key = "AARAV_AND_ISHA" # Needed for part of functionality

#Import functions from Algorithms.py (encapsulation) [Relative import for vercel]
from .Algorithms import time_and_run_binaryKP
from .Algorithms import generate_input_data_binaryKP

@app.route("/", methods=['GET', 'POST'])
def index():
    if request.method == 'POST':
        #Generate data/submit data/load data
        #redirect(url_for('index')) redirets the user back to the homepage (get
method)

        input_data = request.form['input_data']
        if input_data == "generate_random":
            #Calls random data function
            weights, values, capacity =
generate_input_data_binaryKP(int(request.form['number_of_items']),
int(request.form['range_of_values']), float(request.form['correlation_strength']),
float(request.form['capacity_load']))
            elif input_data == "input_manually":
                try:
                    #Strips the input, splits the data by " ", and converts to integer
lists for the 6.2 Input Manually objective
                    weights = list(map(int, request.form['weights'].strip().split()))
                    values = list(map(int, request.form['values'].strip().split()))
                    capacity = int(request.form['capacity'].strip())
                except:
                    session['error'] = "Please enter integers seperated by spaces for
weights and values and a single integer for capacity."
                    return redirect(url_for('index'))
            elif input_data == "import_csv":
```

```

#Gets CSV file, 6.3 Import CSV objective
file = request.files.get("csv_file")
if file and file.filename.endswith(".csv"):
    #Reads CSV file using pandas. Calls any error before converting
    df = pd.read_csv(file)
    #Check file contains individual columns
    if 'weights' not in df.columns:
        session['error'] = "Column 'weights' not found in CSV."
        return redirect(url_for('index'))
    elif 'values' not in df.columns:
        session['error'] = "Column 'values' not found in CSV."
        return redirect(url_for('index'))
    elif 'capacity' not in df.columns:
        session['error'] = "Column 'capacity' not found in CSV."
        return redirect(url_for('index'))
else:
    #File type is not CSV file
    session['error'] = "Not CSV File."
    return redirect(url_for('index'))

try:
    weights = df['weights'].tolist()
    values = df['values'].tolist()
    capacity_list = df['capacity'].tolist()
    #Cleans capacity list so that actual value can be identified
    cleaned_list = [x for x in capacity_list if not math.isnan(x)]
except:
    session['error'] = "Column 'weights', 'values' and 'capacity' all
must contain integer values."
    return redirect(url_for('index'))

if len(cleaned_list) == 0:
    #No value found in capacity
    session['error'] = "No valid (non-NaN) values found in 'capacity'."
    return redirect(url_for('index'))

capacity = int(cleaned_list[0])

if len(weights) != len(values):
    session['error'] = "The number of weights inputted doesn't match the
number of values."
    return redirect(url_for('index'))

# Store results in session or temp storage
session['weights'] = weights
session['values'] = values

```

```

    session['capacity'] = capacity

    session['number_of_items'] = request.form.get('number_of_items', 450)
    session['range_of_values'] = request.form.get('range_of_values', 200)
    session['correlation_strength'] = request.form.get('correlation_strength',
0.6)

    session['capacity_load'] = request.form.get('capacity_load', 0.75)

    # Redirect to GET route
    return redirect(url_for('index'))

# GET request: load stored data or show empty
weights = session.pop('weights', [])
values = session.pop('values', [])
capacity = session.pop('capacity', 0)
error = session.pop('error', "")

number_of_items = session.pop('number_of_items', 450)
range_of_values = session.pop('range_of_values', 200)
correlation_strength = session.pop('correlation_strength', 0.6)
capacity_load = session.pop('capacity_load', 0.75)

#Returns the index.html render template with the newly generated weights, value
and capacity
#Additionally stores the generate random data parameters if new data with same
parameters needs to be generated
    return render_template(
        "index.html",
        weights=weights,
        values=values,
        capacity=capacity,
        number_of_items=number_of_items,
        range_of_values=range_of_values,
        correlation_strength=correlation_strength,
        capacity_load=capacity_load,
        error=error
    )

@app.route('/submit', methods=['POST'])
def submit():
    #.form[''] returns the value of the option from dropdown
    algorithm_type = request.form['algorithm_type']
    function_names = ['Randomised Search', 'Brute Force (iterative)', 'Brute Force
(recursive)', 'Greedy Approximation', 'DP (Memoization)', 'DP (Tabulation)', 'Branch
and Bound', 'Genetic Algorithm', 'Ant Colony Optimisation', 'Simulated Annealing',
'Tabu Search', 'Particle Swarm Optimisation']

```

```

#Gets the index of the algorithm selected
function_index = function_names.index(algorithm_type)
#Efficiently converts string of weights and values into an actual python stored
integer list
weights = ast.literal_eval(request.form['weights'])
values = ast.literal_eval(request.form['values'])
capacity = int(request.form['capacity'])

if len(weights) == 0:
    #No data submitted
    return render_template('index.html', weights=[], values=[], capacity=0,
error="No data value submitted")

#Dictionary of all algorithm parameters. All are grouped together, however only
the selected algorithm's parameters will be used by Algorithm.py
algorithm_data = {
    "time_limit" : int(request.form['time_limit']),
    "max_iterations_iterative" : int(request.form['max_iterations_iterative']),
    "max_iterations_recursive" : int(request.form['max_iterations_recursive']),
    "max_iterations_tabu_search" :
int(request.form['max_iterations_tabu_search']),
    "tabu_list_size" : int(request.form['tabu_list_size']),
    "initial_temperature" : float(request.form['initial_temperature']),
    "minimum_temperature" : float(request.form['minimum_temperature']),
    "alpha" : float(request.form['alpha']),
    "max_iterations_SA" : int(request.form['max_iterations_SA']),
    "max_iterations_ACO" : int(request.form['max_iterations_ACO']),
    "ants" : int(request.form['ants']),
    "alpha_ACO" : int(request.form['alpha_ACO']),
    "beta_ACO" : int(request.form['beta_ACO']),
    "evaporation_rate" : float(request.form['evaporation_rate']),
    "max_iterations_GA" : int(request.form['max_iterations_GA']),
    "population_size_GA" : int(request.form['population_size_GA']),
    "selection_type_GA" : request.form['selection_type_GA'],
    "tournament_size_GA" : int(request.form['tournament_size_GA']),
    "cross_over_rate" : float(request.form['cross_over_rate']),
    "mutation_rate" : float(request.form['mutation_rate']),
    "elitism_rate" : float(request.form['elitism_rate']),
    "max_iterations_PSO" : int(request.form['max_iterations_PSO']),
    "population_size_PSO" : int(request.form['population_size_PSO']),
    "inertia" : float(request.form['inertia']),
    "cognitive_factor" : float(request.form['cognitive_factor']),
    "social_factor" : float(request.form['social_factor']),
    "steepness" : float(request.form['steepness']),
    "max_velocity" : float(request.form['max_velocity']),
    "is_greedy_SA" : True if "is_greedy_SA" in request.form.keys() else False,

```

```

        "is_greedy_tabu" : True if "is_greedy_tabu" in request.form.keys() else False
    }

    #Processes the forms data
    #Calls algorithm on the loaded data
    generated_answer, total_weight, binary_string, elapsed_time, valid, fig =
time_and_run_binaryKP(function_index, weights, values, capacity, algorithm_data)
    if type(fig) == type(numpy.array([0])):
        #If fig is from DP Tabulation, convert to json
        #8.3 DP Tabulation Table objective
        l_fig = fig.tolist()
        r_fig = json.dumps(l_fig)
    else:
        #If fig is a plotted graph, convert using base64 into an image that can be
embedded in the html
        #8.2 Plotted Graph objective
        output = io.BytesIO()
        FigureCanvas(fig).print_png(output)
        r_fig = base64.b64encode(output.getvalue()).decode('utf-8')

    #Dictionary of grouped comparative data that needs to be passed into submit.html
    Data = {
        "approach" : function_names[function_index],
        "total_value" : generated_answer,
        "total_weight" : total_weight,
        "time" : elapsed_time,
        "valid" : valid,
        "binary_string" : binary_string,
        "weights" : weights,
        "values" : values,
        "r_fig" : r_fig
    }

    return render_template("submit.html", Data=Data)

#app.run(debug=True) – This is used in testing not in deployed version on vercel

```

base.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <!--Fixed attributes across index and submit html files-->
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <link rel="stylesheet" href="{{ url_for('static', filename = 'css/main.css')}}">
    <link rel="icon" type="image/x-icon" href="{{ url_for('static',
filename='images/KnapSackTitleImage.png') }}">
    {% block head %}{% endblock %}
</head>
<body>
    {% block body %}{% endblock %}
</body>
</html>
```

index.html

```
{% extends 'base.html' %}

{% block head %}
<title>Knapsack Algorithm Learner Tool</title>

<script>
//Math notation should be displayed on the left of the display and connect MathJax
notation for Latex rendering
window.MathJax = {
  chtml: {
    displayAlign: "left",
    displayIndent: "0"
  }
};
</script>
<script src="https://cdn.jsdelivr.net/npm/mathjax@3/es5/tex-chtml.js"></script>

{% endblock %}

{% block body %}

<div id="title">
  <!--Title and Title Image-->
  
  <h1>A Comparative and Visual Analysis of Exact and Heuristic Algorithms for the
Knapsack Problem</h1>
</div>
<div class="row">
  <div class="column1">
    <div class="algorithm form">
      <form action="/submit" method="post" id="algorithm form"
onsubmit="show_text()">
        <div>
          <div id="problem_description_01">
            <!--0/1 Optimisation version of the knapsack problem 4.1
Problem Description objective -->
            <strong>Problem Description: </strong>
            <p id="problem_description">Given a set of 'n' items where
each item 'i' has a weight and a value ( $w_i$ ,  $v_i$ ) and a knapsack
with a maximum capacity 'W', what subset of items maximises the total value of the
knapsack without exceeding it's weight limit.</p>
            <p id="math_notation">
              \[
              \begin{aligned}
\end{aligned}
\]
```

```

&\text{Maximize} \& \sum_{i=1}^n v_i x_i \\\
&\text{subject to} \& \sum_{i=1}^n w_i x_i \leq W, \\\
& \& x_i \in \{0,1\}, \quad i = 1, \dots, n
\end{aligned}
\\
</p>
</div>
</div>
<strong>Algorithm: </strong>
<br>
<label for="algorithm_type">Choose an algorithm:</label>
<select name="algorithm_type" id="algorithm_type">
  <!--All the available algorithms loosely ordered by exact -->
  heuristic-->
    <option value="Randomised Search">Randomised Search</option>
    <option value="Brute Force (iterative)">Brute Force (iterative)
  </option>
    <option value="Brute Force (recursive)">Brute Force (recursive)
  </option>
    <option value="Greedy Approximation">Greedy
  Approximation</option>
    <option value="DP (Memoization)">DP (Memoization)</option>
    <option value="DP (Tabulation)">DP (Tabulation)</option>
    <option value="Branch and Bound">Branch and Bound</option>
    <option value="Genetic Algorithm">Genetic Algorithm</option>
    <option value="Ant Colony Optimisation">Ant Colony
  Optimisation</option>
    <option value="Simulated Annealing">Simulated Annealing</option>
    <option value="Tabu Search">Tabu Search</option>
    <option value="Particle Swarm Optimisation">Particle Swarm
  Optimisation</option>
  </select>
  <br>
  <div id="0/1algorithms">
    <!--All algorithm descriptions 4.2 Algorithm Descriptions
  objective-->
    <!--Randomised Search-->
    <div class="algorithm" id="Randomised Search">
      <p class="algorithm-description">Uniformly generates random
solutions over a given time period, and stores the best valid solution.</p>
      <!--Example of 5.1 Slider Visual-->
      <div class="variable">
        <label for="time_limit">Search time limit: </label>
        <div class="slidecontainer">
          <input type="range" min="1" max="30" value="2"
class="slider" name="time_limit">

```

```

        </div>
        <input type="number" class="slidevalue" min="1" max="30"
value="2">

        </div>
    </div>
    <!--Brute Force Iterative-->
    <div class="algorithm" id="Brute Force (iterative)"
style="display:none">
        <p class="algorithm-description">Loops in binary from 0 to
2<sup>n-1</sup> (over all possible individual subsets) and stores the best valid
solution.</p>
        <p class="time-complexity">Time Complexity: \(\ \mathcal{O}\{
(2^n) \}</p>
        <div class="variable">
            <label for="max_iterations_iterative">Max iterations:
</label>
            <div class="slidecontainer">
                <input type="range" min="100" max="25000000"
value="200000" class="slider" name="max_iterations_iterative">
            </div>
            <input type="number" class="slidevalue" min="100"
max="25000000" value="200000">
        </div>
    </div>
    <!--Brute Force Recursive-->
    <div class="algorithm" id="Brute Force (recursive)"
style="display:none">
        <p class="algorithm-description">Recursively solves the
problem, by splitting it into smaller sub-problems, using the recursive idea below:
</p>
        <!--Recursive description of the algorithm/problem-->
        <p>
            $$
            k(W, n) =
            \begin{cases}
            0, & \text{if } W \leq 0 \text{ or } n \geq N, \\[6pt]
            k(W, n+1), & \text{if } w_n > W, \\[6pt]
            \max\left(
            \underbrace{v_n + k(W - w_n, n+1)}_{\text{Include
item}}, \right. \\
            \underbrace{k(W, n+1)}_{\text{Exclude item}} \\
            \left. \right), & \text{otherwise.} \\
            \end{cases}
            $$
        </p>
        <p class="time-complexity">Time Complexity: \(\ \mathcal{O}\{

```

(2^n)

label for="max_iterations_recursive">Recursion limit:

Greedy Approximation

style="display:none">

Sorts the items in descending order based upon their value/weight ratio. The algorithm then loops through the list until no more items can be packed.

Time Complexity: $O(n \log n)$

$(n \log n)$

Dynamic Programming Memoization

DP (Memoization)

style="display:none">

Due to the problem having an overlapping sub-problem structure, the memoization technique stems from the recursive approach but caches the results of previous sub-problems, and looks them up before recomputing therefore avoiding redundant calculations.

Time Complexity: $O(nW)$

(nW)

Dynamic Programming Tabulation

DP (Tabulation) style="display:none">

Tabulation creates a table and builds it up starting with resolving the smallest sub-problems until the main problem is solved.

Time Complexity: $O(nW)$

(nW)

Branch and Bound

Branch and Bound

style="display:none">

We use the greedy approach to generate upper and lower bounds for the knapsack problem, and then prune branches to reduce the search space. We always explore the most promising sub-problems first.

```

    </div>
    <!--Simulated Annealing-->
    <div class="algorithm" id="Simulated Annealing"
style="display:none">
        <p class="algorithm-description">Annealing in metallurgy is
when a material is first heated up to a high temperature and then cooled down slowly
over time to achive a stable crystalline structure. The algorithm takes inspiration
from this with the 'temperature' variable corresponding to the degree of randomness
in the search process which will decrease over time to refine the solution. 'Alpha'
is the factor by which the temperature reduces per iteration.</p>
        <label class="checkbox-container">Greedy Initialisation:
            <input name="is_greedy_SA" type="checkbox" checked>
            <span class="checkmark"></span>
        </label>
        <div class="variable">
            <label for="initial_temperature">Initial temperature:
</label>

            <div class="slidecontainer">
                <input type="range" min="1" max="10" step="0.1"
value="3.5" class="slider" name="initial_temperature">
            </div>
            <input type="number" class="slidevalue" min="1" max="10"
step="0.1" value="3.5">
        </div>
        <div class="variable">
            <label for="minimum_temperature">Minimum temperature:
</label>

            <div class="slidecontainer">
                <input type="range" min="0" max="1" step="0.1"
value="0.4" class="slider" name="minimum_temperature">
            </div>
            <input type="number" class="slidevalue" min="0" max="1"
step="0.1" value="0.4">
        </div>
        <div class="variable">
            <label for="alpha">Alpha: </label>
            <div class="slidecontainer">
                <input type="range" min="0.9" max="0.9999"
step="0.0001" value="0.9995" class="slider" name="alpha">
            </div>
            <input type="number" class="slidevalue" min="0.9"
max="0.9999" step="0.0001" value="0.9995">
        </div>
        <div class="variable">
            <label for="max_iterations_SA">Max iterations: </label>
            <div class="slidecontainer">

```

```

        <input type="range" min="1000" max="10000"
value="3300" class="slider" name="max_iterations_SA">
    </div>
    <input type="number" class="slidevalue" min="1000"
max="10000" value="3300">
    </div>
</div>
<!--Tabu Search-->
<div class="algorithm" id="Tabu Search" style="display:none">
    <p class="algorithm-description">Starts with a candidate
solution and repeatedly finds the best neighbouring solution. Stores previously
visited solutions in a tabu list to avoid getting trapped in a local optima.</p>
    <label class="checkbox-container">Greedy Initialisation:
        <input name="is_greedy_tabu" type="checkbox" checked>
        <span class="checkmark"></span>
    </label>
    <div class="variable">
        <label for="max_iterations_tabu_search">Max iterations:
</label>

        <div class="slidecontainer">
            <input type="range" min="250" max="3000" value="1500"
class="slider" name="max_iterations_tabu_search">
            </div>
            <input type="number" class="slidevalue" min="250"
max="3000" value="1500">
        </div>
        <div class="variable">
            <label for="tabu_list_size">Tabu list size: </label>
            <div class="slidecontainer">
                <input type="range" min="10" max="1000" value="500"
class="slider" name="tabu_list_size">
                </div>
                <input type="number" class="slidevalue" min="10"
max="1000" value="500">
            </div>
        </div>
<!--Particle Swarm Optimisation (PSO)-->
<div class="algorithm" id="Particle Swarm Optimisation"
style="display:none">
    <p class="algorithm-description">Inspired by natural life
processes such as the social behaviour of birds flocking or fish schooling, a
population (swarm) of candidate solutions is called particles, where each particle
has a position and velocity value over the search space. Each particle is influenced
by its own best and the global best known solution in order to explore the search
space.</p>

    <div class="variable">

```

```

        <label for="max_iterations_PSO">Max iterations: </label>
        <div class="slidecontainer">
            <input type="range" min="10" max="300" value="55"
class="slider" name="max_iterations_PSO">
        </div>
        <input type="number" class="slidevalue" min="10"
max="300" value="55">
    </div>
    <div class="variable">
        <label for="population_size_PSO">Number of particles:
</label>
        <div class="slidecontainer">
            <input type="range" min="10" max="300" value="35"
class="slider" name="population_size_PSO">
        </div>
        <input type="number" class="slidevalue" min="10"
max="300" value="35">
    </div>
    <div class="variable">
        <label for="inertia">Inertia: </label>
        <div class="slidecontainer">
            <input type="range" min="0" max="1" value="0.8"
step="0.01" class="slider" name="inertia">
        </div>
        <input type="number" class="slidevalue" min="0" max="1"
value="0.8" step="0.01">
    </div>
    <div class="variable">
        <label for="cognitive_factor">Cognitive factor: </label>
        <div class="slidecontainer">
            <input type="range" min="1" max="5" value="1.2"
step="0.1" class="slider" name="cognitive_factor">
        </div>
        <input type="number" class="slidevalue" min="1" max="5"
value="1.2" step="0.1">
    </div>
    <div class="variable">
        <label for="social_factor">Social factor: </label>
        <div class="slidecontainer">
            <input type="range" min="1" max="5" value="3"
step="0.1" class="slider" name="social_factor">
        </div>
        <input type="number" class="slidevalue" min="1" max="5"
value="3" step="0.1">
    </div>
    <div class="variable">

```

```

        <label for="steepness">Sigmoid function steepness:
</label>

        <div class="slidecontainer">
            <input type="range" min="1" max="3" value="2.2"
step="0.1" class="slider" name="steepness">
        </div>
        <input type="number" class="slidevalue" min="1" max="3"
value="2.2" step="0.1">
    </div>
    <div class="variable">
        <label for="max_velocity">Maximum velocity: </label>
        <div class="slidecontainer">
            <input type="range" min="1" max="10" value="4.3"
step="0.1" class="slider" name="max_velocity">
        </div>
        <input type="number" class="slidevalue" min="1" max="10"
value="4.3" step="0.1">
    </div>
</div>
<!--Genetic Algorithm (GA)-->
<div class="algorithm" id="Genetic Algorithm"
style="display:none">
    <p class="algorithm-description">Simulates natural selection
across multiple candidate solutions. Each generation, crossover between solutions and
mutation occurs. Selection techniques combined with elitism are used to pick the best
solutions for the next generation. This process repeats over a large number of
generations, keeping track of the best valid solution.</p>
    <div class="variable">
        <label for="max_iterations_GA">Generations: </label>
        <div class="slidecontainer">
            <input type="range" min="50" max="300" value="100"
class="slider" name="max_iterations_GA">
        </div>
        <input type="number" class="slidevalue" min="50"
max="300" value="100">
    </div>
    <div class="variable">
        <label for="population_size_GA">Population size: </label>
        <div class="slidecontainer">
            <input type="range" min="10" max="300" value="50"
class="slider" name="population_size_GA">
        </div>
        <input type="number" class="slidevalue" min="10"
max="300" value="50">
    </div>
    <label for="selection_type_GA">Selection type:</label>

```

```

        <select name="selection_type_GA" id="selection_type_GA">
            <option value="Tournament selection">Tournament
selection</option>
            <option value="Roulette wheel selection">Roulette wheel
selection</option>
        </select>
        <div class="variable" id="tournament_size_variable">
            <label for="tournament_size_GA">Tournament size: </label>
            <div class="slidecontainer">
                <input type="range" min="1" max="20" value="9"
class="slider" name="tournament_size_GA">
            </div>
            <input type="number" class="slidevalue" min="1" max="20"
value="9">
        </div>
        <div class="variable">
            <label for="cross_over_rate">Cross over rate: </label>
            <div class="slidecontainer">
                <input type="range" min="0" max="1" value="0.6"
step="0.01" class="slider" name="cross_over_rate">
            </div>
            <input type="number" class="slidevalue" min="0" max="1"
value="0.6" step="0.01">
        </div>
        <div class="variable">
            <label for="mutation_rate">Mutation rate: </label>
            <div class="slidecontainer">
                <input type="range" min="0" max="0.2" value="0.05"
step="0.01" class="slider" name="mutation_rate">
            </div>
            <input type="number" class="slidevalue" min="0"
max="0.2" value="0.05" step="0.01">
        </div>
        <div class="variable">
            <label for="elitism_rate">Elitism rate: </label>
            <div class="slidecontainer">
                <input type="range" min="0" max="0.2" value="0.05"
step="0.01" class="slider" name="elitism_rate">
            </div>
            <input type="number" class="slidevalue" min="0" max="0.2"
value="0.05" step="0.01">
        </div>
    </div>
    <!--Ant Colony Optimisation (ACO)-->
    <div class="algorithm" id="Ant Colony Optimisation"
style="display:none">

```

`<p class="algorithm-description">ACO is inspired by the behaviour of ants when searching for food. Each ant lays down a pheromone trail (a type of hormone) when exploring a particular path. Over time, good/frequently chosen paths will accumulate pheromone, attracting more ants. Additionally, the choices a 'path' takes is influenced by other heuristic information (attractiveness). Evaporation of pheromone occurs, so that older suboptimal paths lose influence over time.</p>`

```

    <div class="variable">
        <label for="max_iterations_ACO">Max iterations: </label>
        <div class="slidecontainer">
            <input type="range" min="1" max="100" value="11"
class="slider" name="max_iterations_ACO">
        </div>
        <input type="number" class="slidevalue" min="1" max="100"
value="11">
    </div>
    <div class="variable">
        <label for="ants">Number of ants: </label>
        <div class="slidecontainer">
            <input type="range" min="1" max="20" value="5"
class="slider" name="ants">
        </div>
        <input type="number" class="slidevalue" min="1" max="20"
value="5">
    </div>
    <div class="variable">
        <label for="alpha_ACO">Alpha: </label>
        <div class="slidecontainer">
            <input type="range" min="1" max="100" value="5"
class="slider" name="alpha_ACO">
        </div>
        <input type="number" class="slidevalue" min="1" max="100"
value="5">
    </div>
    <div class="variable">
        <label for="beta_ACO">Beta: </label>
        <div class="slidecontainer">
            <input type="range" min="1" max="100" value="1"
class="slider" name="beta_ACO">
        </div>
        <input type="number" class="slidevalue" min="1" max="100"
value="1">
    </div>
    <div class="variable">
        <label for="evaporation_rate">Evaporation rate:
</label>
```

```

        <div class="slidecontainer">
            <input type="range" min="0" max="1" step="0.01"
value="0.9" class="slider" name="evapouration_rate">
        </div>
        <input type="number" class="slidevalue" min="0" max="1"
step="0.01" value="0.9">
    </div>
</div>
<!--Used to store data values within the page (this is hidden from
the user)-->
<div id="data" style="display:none">
    <textarea name="weights">{{ weights }}</textarea>
    <textarea name="values">{{ values }}</textarea>
    <textarea name="capacity">{{ capacity }}</textarea>
</div>
</form>
</div>
<br>
<strong>Data: </strong>
<div class="data form">
    <!--Form initially submitted to generate/submit/upload data-->
    <form method="post" id="data form" enctype="multipart/form-data" action =
"/">

        <label for="input_data">Input data:</label>
        <select name="input_data" id="input_data">
            <option value="generate_random">Generate random</option>
            <option value="input_manually">Input manually</option>
            <option value="import_csv">Import CSV</option>
        </select>
        <div id="generate_random">
            <!-- This will having the following control variables: Number of
items, Range of values, Correlation strength, Capacity Load-->
            <div class="variable">
                <label for="number_of_items">Number of items: </label>
                <div class="slidecontainer">
                    <input type="range" min="5" max="1000" value="{{
number_of_items }}" class="slider" name="number_of_items">
                </div>
                <input type="number" class="slidevalue" min="5" max="1000"
value="{{ number_of_items }}">
            </div>
            <div class="variable">
                <label for="range_of_values">Range of values: </label>
                <div class="slidecontainer">
                    <input type="range" min="10" max="1000" value="{{

```

```

range_of_values }}" class="slider" name="range_of_values">
    </div>
    <input type="number" class="slidevalue" min="10" max="1000"
value="{{ range_of_values }}">
    </div>
    <div class="variable">
        <label for="correlation_strength">Correlation strength:
</label>

        <div class="slidecontainer">
            <input type="range" min="0" max="1" value="{{
correlation_strength }}" class="slider" name="correlation_strength" step="0.01">
            </div>
            <input type="number" class="slidevalue" min="0" max="1"
value="{{ correlation_strength }}" step="0.01">
            </div>
            <div class="variable">
                <label for="capacity_load">Capacity load: </label>
                <div class="slidecontainer">
                    <input type="range" min="0.1" max="0.9" value="{{
capacity_load }}" class="slider" name="capacity_load" step="0.01">
                    </div>
                    <input type="number" class="slidevalue" min="0.1" max="0.9"
value="{{ capacity_load }}" step="0.01">
                    </div>
                </div>
                <div id="input_manually" style="display:none">
                    <br>
                    <label for="weights">Enter the weights: </label>
                    <input type="text" name="weights" placeholder="17 20 87 54 34">
                    <br>
                    <label for="values">Enter the values: </label>
                    <input type="text" name="values" placeholder="20 24 12 87 45">
                    <br>
                    <label for="capacity">Enter the capacity:</label>
                    <input type="text" name="capacity" placeholder="135">
                    <br>
                </div>
                <div id="import_csv" style="display:none">
                    <br>
                    <p>You can upload a .csv file as input data when layed out in the
following format: columns ('weights', 'values', 'capacity')</p>
                    <input type="file" name="csv_file" accept=".csv">
                    <p style="color: blue;">
                        Choose csv file to upload
                    </p>
                </div>

```

```

        </form>
    </div>
</div>
<!--Capacity marker and scrollbar-->
<div class="column2">
    <strong>Capacity:</strong> {{ capacity }}
    <br>
    <strong>Items (w<sub>i</sub>, v<sub>i</sub>):</strong>
    <div class="scroll-box">
        <!--Shows all the items loaded into the website in the vertical
scrollbox-->
        {% for i in range(weights|length) %}
            <div class="item">Item {{ i + 1 }}: ({{ weights[i] }}, {{ values[i]
}}})</div>
        {% endfor %}
    </div>
</div>
<br>
<div class="button_controls">
    <button type="submit" form="data form" id="generate_data_button">Generate
data</button>
    <button type="submit" form="algorithm form">Run</button>
</div>
<div style="display:none" id="waiting_text">
    <!--Running algorithm message is displayed to the user as some algorithms may
take a significant time to load-->
    <p class="message-text" >Running algorithm...</p>
</div>
<!--Error handling (Specific error message is inserted below)-->
{% if error %}
<p class="message-text">{{ error }}</p>
{% endif %}
<br>
<script>
    //Used to dynamically update the menus

    //Stops submission on enter key press due to their being 2 forms
    document.getElementById("algorithm form").addEventListener("keydown",
function(event) {
    if (event.key === "Enter"){
        event.preventDefault()
    }
});

    document.getElementById("data form").addEventListener("keydown", function(event)

```

```

{
    if (event.key === "Enter"){
        event.preventDefault()
    }
});

//Matches the algorithm parameters to the algorithm selected
function change_algorithm_parameters () {

document.getElementById("0/1algorithms").querySelectorAll(".algorithm").forEach(algorithm
=> {
    if (algorithm_setting.id === dropdown_binary_algorithm.value) {
        algorithm_setting.style.display = "inline";
    }
    else {
        algorithm_setting.style.display = "none";
    }
});
}

const dropdown_binary_algorithm = document.getElementById("algorithm_type");
dropdown_binary_algorithm.addEventListener("change",
change_algorithm_parameters);

//Manually uses switch statement in order to match the form parameters for the
input data type
const dropdown_input_data = document.getElementById("input_data");
const generate_random_menu = document.getElementById("generate_random");
const input_manually_menu = document.getElementById("input_manually");
const import_csv_menu = document.getElementById("import_csv");
const generate_data_button = document.getElementById("generate_data_button");

dropdown_input_data.addEventListener("change", function() {
    generate_random_menu.style.display = "none";
    input_manually_menu.style.display = "none";
    import_csv_menu.style.display = "none";

    if (this.value === 'generate_random') {
        generate_random_menu.style.display = "inline";
        generate_data_button.innerHTML = "Generate data";
    }
    else if (this.value === 'input_manually') {
        input_manually_menu.style.display = "inline";
        generate_data_button.innerHTML = "Submit data";
    }
    else if (this.value === 'import_csv') {

```

```

import_csv_menu.style.display = "inline";
generate_data_button.innerHTML = "Upload csv";
    }
});

//Specific menus for genetic algorithms
const selection_type_GA = document.getElementById("selection_type_GA");
const tournament_size_variable =
document.getElementById("tournament_size_variable");
selection_type_GA.addEventListener("change", function() {
    if (this.value === "Tournament selection") {
        tournament_size_variable.style.visibility = "initial";
    }
    else if (this.value === "Roulette wheel selection") {
        tournament_size_variable.style.visibility = "hidden";
    }
});

//Sync slider and textbox value
document.querySelectorAll(".variable").forEach(group => {
    const slider =
group.querySelector(".slidecontainer").querySelector("input[type=range]");
    const number_input = group.querySelector(".slidevalue");

    // Update number when slider changes
    slider.addEventListener('input', function() {
        number_input.value = slider.value;
    });

    // Update slider when number changes (with validation)
    number_input.addEventListener('input', function() {
        let value = number_input.value;
        //Clamps the value between slider.min and slider.max
        if (value < Number(slider.min)) value = slider.min;
        if (value > Number(slider.max)) value = slider.max;
        slider.value = value;
        number_input.value = value;
    });
});

const waiting_text = document.getElementById("waiting_text");

function show_text() {
    waiting_text.style.display = "inline";
}

```

```
// Run when DOM is fully loaded (initial load)
document.addEventListener("DOMContentLoaded", function () {
    runOnLoadOrBack();
});

// Run when page is shown
window.addEventListener("pageshow", function (event) {
    if (event.persisted || performance.getEntriesByType("navigation")[0].type ===
"back_forward") {
        runOnLoadOrBack();
    }
});

//Hides the waiting text
function runOnLoadOrBack() {
    waiting_text.style.display = "none";
    change_algorithm_parameters();
}

</script>

{% endblock %}
```

submit.html

```
{% extends 'base.html' %}

{% block head %}
<title>Submitted</title>
{% endblock %}

{% block body %}
<h1>{{ Data['approach'] }} (Result Data)</h1> <!--Name of the algorithm-->
<div class="row">
    <div class="column3">
        <div id="result_data">
            <strong>Value: </strong> {{ Data['total_value'] }} <!--Value of solution-->
            <br>
            <strong>Weight: </strong> {{ Data['total_weight'] }} <!--Weight of solution-->
            <br>
            <strong>Valid: </strong> {{ Data['valid'] }} <!--As a debugging check to test if the solution is valid-->
            <br>
            <strong>Time(s): </strong> {{ Data['time'] }} <!--Execution time-->
            <br>
            <strong>Items (w<sub>i</sub>, v<sub>i</sub>) included: </strong>
            <br>
            <div class="scroll-box">
                <!--All items in binary_string that are '1'-->
                {% for i in range(Data['binary_string']|length) %}
                    {% if Data['binary_string'][i] == '1': %}
                        <div class="item">Item {{ i + 1 }}: ({{ Data['weights'][i]
}}, {{ Data['values'][i] }})</div>
                    {% endif %}
                {% endfor %}
            </div>

            <strong>Items code: </strong>
            <br>
            <div class="scroll-box-mini">
                <!--Actual binary string returned by program-->
                <div class="item">{{ Data['binary_string'] }}</div>
            </div>
        </div>
    </div>
    <div class="column4">
        <!--Change display output if it is plotted graph or DP table-->
```

```

    {% if Data['approach'] == 'DP (Tabulation)': %}
        <!--DP table description-->
        <p>Below shows the first 250(Capacity/Columns) x 250(Items Included/Rows)
in the dp table as previously explained in the algorithm:</p>
        <div id="canvas-container">
            <canvas id="dp-canvas"></canvas>
        </div>
    {% else: %}
        <div id="graphs">
            <!--Insert encoded image-->
            
            <!--Download image linked to the same source-->
            <a id="download_button" href="data:image/png;base64,{{ Data['r_fig']
}}> download>Download Image</a>
        </div>
    {% endif %}
</div>
<br>
<script>
    //parse json file
    const dpData = JSON.parse('{{ Data["r_fig"]|safe }}');

    const cellWidth = 40; //width of cell
    const cellHeight = 20; //height of cell

    //rows and cols may not be 250 if not maximised
    const rows = dpData.length;
    const cols = dpData[0].length;

    const container = document.getElementById("canvas-container");
    const canvas = document.getElementById("dp-canvas");
    const ctx = canvas.getContext("2d");

    //Set virtual canvas size
    canvas.width = Math.min(cellWidth * cols, 10000);
    canvas.height = Math.min(cellHeight * rows, 5000);

    //console.log(canvas.width, canvas.height)

    //Function that draws the current displayed output in the container
    function drawVisible() {

        const scrollTop = container.scrollTop;
        const scrollLeft = container.scrollLeft;

```

```

    const startRow = Math.floor(scrollTop / cellHeight);
    const endRow = Math.min(rows, startRow + Math.ceil(container.clientHeight /
cellHeight) + 1);

    const startCol = Math.floor(scrollLeft / cellWidth);
    const endCol = Math.min(cols, startCol + Math.ceil(container.clientWidth /
cellWidth) + 1);

    //clear output before drawing cells
    ctx.clearRect(0, 0, canvas.width, canvas.height);

    //Output text format settings
    ctx.font = "12px Arial";
    ctx.textAlign = "center";
    ctx.textBaseline = "middle";

    for (let r = startRow; r < endRow; r++) {
        for (let c = startCol; c < endCol; c++) {
            //x, y coordinates of the cell
            const x = c * cellWidth;
            const y = r * cellHeight;

            //Draw the cells with appropriate colours

            if (r === 0 || c === 0)
            {
                ctx.fillStyle = "#fff";
                //Coloured Rectangle
                ctx.fillRect(x, y, cellWidth, cellHeight);
            }

            ctx.strokeStyle = "#000";
            //Bordered Rectangle
            ctx.strokeRect(x, y, cellWidth, cellHeight);

            ctx.fillStyle = "#000";
            if (!(r === 0 && c === 0))
            {
                //Fill text with dp array
                ctx.fillText(dpData[r][c], x + cellWidth / 2, y + cellHeight /
2);
            }
            else
            {
                //Top left cell should be unmarked
                ctx.fillText('#', x + cellWidth / 2, y + cellHeight / 2)

```

```
        }  
    }  
}  
  
    container.addEventListener("scroll", drawVisible);  
    drawVisible();  
</script>  
</body>  
</html>  
  
{% endblock %}
```

main.css

```
/* Fixed Title Image*/
#title img{
    width: 160px;
    height: 150px;
    margin: 10px;
}

html {
    zoom: 1.25;
}

body {
    margin: 10px;
    font-family: sans-serif;
    line-height: 2;
    background-color: rgb(195, 223, 240);
}

#title {
    display: flex;
    gap: 20px;
}

select {
    width: 200px;
    height: 25px;
    line-height: normal;
}

p, h1, option {
    line-height: normal;
}

h1 {
    margin-bottom: 10px;
}

.variable {
    display: flex;
    gap: 7px;
}

.slidecontainer {
    width: 150px; /* Width of the outside container */
```

```
}

.variable input[type="number"] {
  border: none;
  color: green;
  margin: 3px;
  border-radius: 3px;
}

/* checkbox*/

.checkbox-container {
  display: inline-block;
  position: relative;
  margin: 10px 0;
  cursor: pointer;
  font-size: 16px;
  user-select: none;
}

/* Hide the default checkbox */
.checkbox-container input {
  display: none;
}

/* Custom checkbox square */
.checkmark {
  position: absolute;
  top: 5px;
  left: 150px;
  height: 20px;
  width: 20px;
  background-color: #eee;
  border: 2px solid #ccc;
  border-radius: 4px;
  transition: all 0.2s ease;
  transform: scale(0.8);
}

.checkbox-container:hover input ~ .checkmark {
  background-color: #ddd;
}

/* Checked state */
.checkbox-container input:checked ~ .checkmark {
```

```
background-color: #4CAF50;
border-color: #4CAF50;
}

.checkmark:after {
  content: "";
  position: absolute;
  display: none;
}

.checkbox-container input:checked ~ .checkmark:after {
  display: block;
}

/* Tick styling */
.checkbox-container .checkmark:after {
  left: 6px;
  top: 2px;
  width: 5px;
  height: 10px;
  border: solid white;
  border-width: 0 2px 2px 0;
  transform: rotate(45deg);
}

/* The slider itself */
.slider {
  -webkit-appearance: none;
  width: 100%;
  height: 10px;
  border-radius: 3px;
  background: #d3d3d3;
  outline: none;
  opacity: 0.7;
  -webkit-transition: .2s;
  transition: opacity .2s;
}

.slider::-webkit-slider-thumb {
  -webkit-appearance: none;
  appearance: none;
  width: 12px;
  height: 12px;
  border-radius: 50%;
  background: green;
  cursor: pointer;
}
```

```
}

.slider::-moz-range-thumb {
    width: 12px;
    height: 12px;
    border-radius: 50%;
    background: green;
    cursor: pointer;
}

.message-text {
    color: red;
}

.variable label {
    display: inline-block;
    white-space: nowrap;
    width: auto;
    max-width: 200px;
}

button {
    background-color: rgb(51, 197, 51);
    border-radius: 5px;
    color: white;
    height: 25px;
    width: auto;
    border: none;
    transform: scale(1.15);
}

input[type="file"] {
    background-color: rgb(51, 197, 51);
    color: white;
    padding: 4px 8px;
    border: none;
    border-radius: 5px;
    cursor: pointer;
    margin: 5px;
}

/* vertical scrollbar */
.scroll-box {
    border: 1px solid #ccc;
    padding: 10px;
    max-width: 200px;
}
```

```
width: auto;
height: 300px;
overflow-y: scroll;
background-color: #f9f9f9;
border-radius: 3px;
}

/* horizontal scrollbar */
.scroll-box-mini {
  border: 1px solid #ccc;
  padding: 10px;
  max-width: 200px;
  width: auto;
  height: 45px;
  overflow-x: scroll;
  background-color: #f9f9f9;
  border-radius: 3px;
}

.row {
  display: flex;
  gap: 15px;
}

.column1 {
  width: 75%;
}

.column2 {
  width: 25%;
  padding: 10px;
}

.column3 {
  width: 25%;
}

.column4 {
  width: 75%;
}

.button_controls {
  display: flex;
  gap: 20px;
  margin-left: 15px;
}
```

```
#graphs img {
    max-width: 100%;
    max-height: 100%;
    display: block;
    margin: auto;
    border-radius: 5px;
}

#graphs {
    width: 90%;
    margin-left: auto;
    margin-right: auto;
    text-align: right;
}

#download_button {
    display: inline-block;
    padding: 0px 4px;
    background-color: #007BFF;
    color: white;
    text-decoration: none;
    border-radius: 5px;
    font-family: Arial, sans-serif;
    font-size: 16px;
    transition: background-color 0.3s ease;
    border: 5px;
    margin-top: 5px;
}

#download_button:hover {
    background-color: #0056b3; /* Darker on hover */
}

/* table */

#canvas-container {
    width: 100%;
    height: 500px;
    overflow: auto;
    position: relative;
}

canvas {
    position: absolute;
    top: 0;
```

```
    left: 0;
}

.time-complexity {
    font-style: italic;
}

/* 893 px width aspect ratio changes */

@media (max-width: 893px) {
    body {
        margin: none;
        margin-left: 5px;
    }

    #title {
        flex-direction: column-reverse;
        align-items: center;    /* Vertical */
        text-align: center;
    }

    .row {
        flex-direction: column;
    }

    .column1, .column2, .column3, .column4 {
        width: 100%;
        padding: 10px;
    }

    #title h1 {
        margin-bottom: 0;
    }

    #title img {
        width: 192px;
        height: 180px;
    }

    #graphs {
        width: 96%;
        margin: 0px;
    }
}
```

requirements.txt

```
blinker==1.9.0
cachelib==0.13.0
click==8.3.1
colorama==0.4.6
contourpy==1.3.2
cyclcr==0.12.1
Flask==3.1.3
fonttools==4.61.1
itsdangerous==2.2.0
Jinja2==3.1.6
kiwisolver==1.4.9
MarkupSafe==3.0.3
matplotlib==3.10.8
msgspec==0.20.0
numpy==2.2.6
packaging==26.0
pandas==2.3.3
pillow==12.1.1
pyparsing==3.3.2
python-dateutil==2.9.0.post0
pytz==2025.2
six==1.17.0
tzdata==2025.3
Werkzeug==3.1.6
```

vercel.json

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/api/index.py" }
  ]
}
```